

Automated Sim-to-Real Deployment Framework for Reinforcement Learning Control on Resource-Constrained Microcontrollers

Nidharshan V.

Department of Computer Science & Engineering
University of Moratuwa
Moratuwa, Sri Lanka
nidharshan.23@cse.mrt.ac.lk

Keywords—Sim-to-Real Transfer, Reinforcement Learning, Microcontroller Deployment, TinyML, Dynamic Self-Righting.

I. INTRODUCTION

Reinforcement learning (RL) can teach robots difficult skills, such as walking or getting up after a fall [1], [2]. The robot learns in a physics simulator, and the trained controller (the *policy*) is then moved to the real robot. For large robots this last step is well supported: tools such as Isaac Lab/Orbit [3] and Legged Gym [2] expect a powerful on-board computer running Linux and ROS 2.

Small, low-cost robots are different. They use a microcontroller (MCU) such as the ESP32 (240 MHz, less than 1 MB of RAM, no operating system). Today, putting a trained policy on such a board is done by hand: the network weights are copied into C arrays and the sensor and motor code is written from scratch. Every change to the robot or the network means rewriting this code, and mistakes are easy to make.

We built *simtoreal*, an open-source Python library that does this step automatically. Our contributions are: (1) a converter that turns a trained Stable-Baselines3 or PyTorch policy into MicroPython bytecode (.mpy, the default), MicroPython source or a C header; (2) a generator that writes the complete on-board control loop (*main.py*) from a short YAML description of the hardware; (3) a test suite that checks the generated code against the original policy on a PC, in simulation and on a real ESP32; and (4) a test robot, *Robo1*, with a MuJoCo model built from its CAD parts.

II. LITERATURE REVIEW

Sim-to-real transfer: The difference between simulation and the real world is called the *reality gap*. It is usually reduced by training under many random physical settings (domain randomization, DR) and by measuring the real robot's properties (system identification) [4], [5]. Tan et al. [5] randomized mass, delay and friction to make a four-legged robot walk; Hwangbo et al. [1] learned a model of the motors. GPU simulation [2], [3] made large DR training fast, but the resulting policies run on ONNX or TorchScript, which need tens of megabytes of memory – far more than an MCU has.

Machine learning on microcontrollers: TensorFlow Lite for Microcontrollers (TFLM) [6] and MCUNet [7] run neural networks on MCUs, but they are made mainly for image and speech tasks and use an interpreter with a reserved memory block (the “tensor arena”), not for small control networks. RLtools [8], a C++17 library, can even *train* RL policies on an MCU, but the robot's environment must be rewritten in its own C++ system, so a policy trained with the usual Python tools (Gymnasium, Stable-Baselines3, PyTorch) cannot be reused directly. *simtoreal* keeps these Python tools and automates only the last step.

Measuring the reality gap: There is still no common benchmark for the sim-to-real gap on small embedded devices; Zhou et al. [9] found a large drop in performance when an agent trained only in simulation was run on cheap real hardware. We therefore test each step of our pipeline separately: is the generated code the same as the original, does it behave the same in a closed loop, and what does it cost on the device?

III. MATERIALS AND METHODS

1) Taking out the policy: The library opens a trained Stable-Baselines3 PPO file (.zip) or a PyTorch file (.pt). It keeps only the actor network (the part that chooses the actions): its weights $\mathbf{W}_l$, biases $\mathbf{b}_l$ and activation type. The critic, the optimizer state and other training data are dropped.

2) Generating the network code: For MicroPython, the network is written as a Python module and compiled on the PC into .mpy bytecode with *mpy-cross*. This is the default, because compiling the ~60 kB source on the board itself can run out of memory. For C, the weights become constant arrays stored in Flash, and a *policy_forward()* function passes the data through the layers using two fixed-size buffers that swap roles after each layer. The C code never uses dynamic memory (*malloc*), so its memory use is known in advance.

3) Generating the control loop: The user describes the hardware in one *config.yaml* file. From it, the library writes *main.py*, which creates the sensor and motor drivers (e.g. an MPU-6050 IMU over I2C and SG90 servos), converts every reading to the units used in training (*scale*, *offset*), can give the policy the current value of an action as an extra input,

applies each action directly or as a small step (delta mode) within limits, and repeats the read-decide-act loop at a fixed rate.

4) Test robot (Robo1): *Robo1* (Fig. 1) is a small robot that stands itself up after a fall. It has an ESP32, an MPU-6050 IMU and two SG90 servos. Its MuJoCo model uses the CAD shapes and measured masses of the real parts (94.8 g in total), a joint range of $\pm 90^\circ$, a torque limit of ± 1.0 N·m and a damping of 0.02 N·m·s/rad. The policy only uses values the robot can measure itself: $\mathbf{o}_t = [\phi_{\text{roll}}, \theta_{\text{pitch}}, a_z, q_1, q_2]$, where the vertical acceleration a_z tells upright from upside-down and q_1, q_2 are the current servo targets. Each action moves a servo target by at most 0.08 rad, 50 times per second. To test robustness, we randomize the body mass ($\pm 15\%$), joint damping and friction ($\pm 20\%$) and servo strength ($\pm 15\%$, like a weak battery), and add sensor noise ($\sigma = 0.015$ rad).

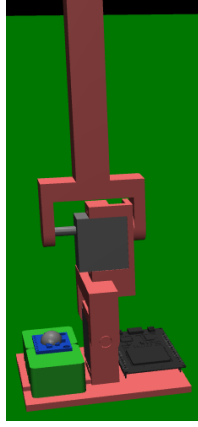


Fig. 1. Robo1 in the MuJoCo simulator.

IV. RESULTS AND DISCUSSION

Is the generated code correct? Our test suite (`tests/`, all results saved as JSON) runs the library exactly as a user would; Table I lists the results. The trained actor is a small network with 5 inputs, two hidden layers of 64 neurons (tanh) and 2 outputs: 4,674 numbers, or 18.7 kB as float32. (i) *Same output on a PC*: we gave 4,609 inputs (2,000 random, 2,609 taken from simulated runs) to the original PyTorch network and to both generated versions. The largest difference was 1.2×10^{-6} for MicroPython and 2.9×10^{-6} for C – only floating-point rounding – and the final actions agreed in 100% of cases. This test also found a bug: the generated C header did not compile for any network with more than one layer. After a one-line fix it compiles with `gcc -Wall -Wextra` and no warnings. (ii) *Same behaviour in simulation*: the generated C code controlled the robot in 1,000 MuJoCo runs (5 start poses $\times$ 50 runs $\times$ normal and randomized physics). Success rate and time to stand up were identical to the PyTorch policy in every case. (iii) *Same output on the ESP32*: on an ESP32 running MicroPython 1.28, the generated `.mpy` matched PyTorch for 20 test inputs.

TABLE I
TEST RESULTS FOR THE CODE GENERATED BY SIMTOREAL

Test	Code	Result
Same output, PC ($n=4,609$)	MicroPython	max. diff. 1.2×10^{-6}
Same output, PC ($n=4,609$)	C header	max. diff. 2.9×10^{-6}
Simulation, 1,000 runs	C header	97.2% success (= PyTorch)
Same output, ESP32 ($n=20$)	MicroPython	max. diff. 9.5×10^{-7}
Memory on ESP32	MicroPython	119.6 kB (72% of heap)
Time per decision, ESP32	MicroPython	155.2 ± 7.4 ms ($7.8 \times$ too slow)
IMU noise, sensor still	MPU-6050	1.6 mrad (simulator: 15 mrad)
Full loop on ESP32	MicroPython	5.4 Hz; I/O 4.2 ms, policy 182 ms
<code>main.py</code> checks	generator	6/6 pass (0/4 before fix)
<code>main.py</code> on ESP32	generator	100/100 cycles, no errors

Cost on the ESP32: The MicroPython policy file is 75.2 kB, but once loaded it uses 119.6 kB of the 162.3 kB free memory – 6.4 times the size of the weights – because MicroPython

stores every number as a separate object. One decision takes 155.2 ± 7.4 ms (200 runs). The policy was trained to decide every 20 ms (50 Hz), so it is 7.8 times too slow and the loop can run at only about 6.4 Hz. As every action is a small step, the robot would move about 7.8 times slower than in training, which changes the behaviour it learned. The MicroPython version is therefore correct, but not fast enough for this policy.

Real sensor and servos: We connected the IMU and both servos to the ESP32 (without the robot body). In a 500-cycle test, reading the IMU took 3.2 ms and moving the servos 0.95 ms – together only 21% of the 20 ms budget – while the policy took 182 ms (98% of each cycle; more than on its own because of memory clean-up), so the loop ran at 5.4 Hz. With the sensor lying still, its noise was 1.3–1.6 mrad (0.2 mrad after filtering), about 10 times less than the 15 mrad added in training, so the simulated sensor was pessimistic. The first generated `main.py` could not run the robot: it failed all four of our interface checks (pin numbers written as text, which the ESP32 rejects; only 2 of the 5 inputs; degrees instead of radians; and the raw network output sent straight to the servo). Adding a_z to the YAML file alone did not help, so we improved the generator (Section III). The new `main.py` passes all six checks, including one for a different, made-up robot, and ran unchanged on the ESP32 for 100 cycles (5.8 Hz, again limited by the network).

Simulation results: Over 50 simulated runs per start pose, the robot stood up from all four standard fallen poses in 100% of runs (in 1.36, 1.00, 1.26 and 0.78 s) and from random falls, including upside-down, in 86% of runs: 97.2% overall, in 1.15 s on average. With randomized physics, success stayed at 97.2%, but standing up took 20% longer (1.38 s) – something to check on the real robot.

V. CONCLUSION

`simtoREAL` turns trained Stable-Baselines3/PyTorch policies into MicroPython or C code for microcontrollers and writes the control loop from a YAML file. Our tests show that the generated code gives the same results as the original policy on a PC, in 1,000 simulated runs and on a real ESP32, and they found bugs in both the C and the control-loop generators, which we fixed. On the real hardware, reading the sensor and moving the servos fit easily into the 50 Hz budget, but the MicroPython network is about 9 times too slow. Next, we will make the MicroPython network faster or use the C header, and run self-righting tests on the assembled robot to measure the real sim-to-real gap.

REFERENCES

- [1] J. Hwangbo *et al.*, “Learning agile and dynamic motor skills for legged robots,” *Sci. Robot.*, vol. 4, no. 26, p. eaa5872, 2019.
- [2] N. Rudin, D. Hoeller, P. Reist, and M. Hutter, “Learning to walk in minutes using massively parallel deep RL,” in *Proc. CoRL, PMLR*, vol. 164, 2022, pp. 91–100.
- [3] M. Mittal *et al.*, “Orbit: A unified simulation framework for interactive robot learning environments,” *IEEE RA-L*, vol. 8, no. 6, pp. 3740–3747, 2023.
- [4] J. Tobin *et al.*, “Domain randomization for transferring deep neural networks from simulation to the real world,” in *Proc. IEEE/RSJ IROS*, 2017, pp. 23–30.
- [5] J. Tan *et al.*, “Sim-to-real: Learning agile locomotion for quadruped robots,” in *Proc. RSS*, 2018.
- [6] R. David *et al.*, “TensorFlow Lite Micro: Embedded ML on TinyML systems,” in *Proc. MLSys*, vol. 3, 2021, pp. 800–811.
- [7] J. Lin *et al.*, “MCUNet: Tiny deep learning on IoT devices,” in *Proc. NeurIPS*, 2020, pp. 11711–11722.
- [8] J. Eschmann, D. Albani, and G. Loanno, “RLtools: A fast, portable deep RL library for continuous control,” *J. Mach. Learn. Res.*, vol. 25, pp. 1–19, 2024.
- [9] R. Zhou, O. Tavalalaie, S. Chen, and A. Y. Zomaya, “Measure the sim-to-real gap: An affordable benchmark platform for RL in AIoT systems,” arXiv:2607.10309, 2026.